Getting Data Ready: Cleaning, Transforming, and Exploring at Scale

BDAT 602 – Data Mining and Applications — Lecture 2

Eric Ocran, PhD

Department of Statistics and Actuarial Science
University of Ghana

Outline

The Dirty Secret of Data Mining

The 80/20 Rule of Data Science

In real-world projects, data scientists spend approximately **80% of their time** cleaning and preparing data, and only **20% actually mining it**.

The Dirty Secret of Data Mining

Why data is never clean in practice:

- Data was collected for a *different* purpose than analysis.
- Multiple systems with inconsistent formats were merged.
- Human data entry introduces errors and inconsistencies.

Consequences of skipping preparation:

- **Association rules** discover spurious patterns driven by missing-value artefacts.
- ***k*-means** clusters are distorted by unscaled variables (income dominates BMI).

The Dirty Secret of Data Mining

Why data is never clean in practice:

- Sensors and automated systems fail or produce outliers
- Privacy regulations mask or truncate sensitive fields
- Time pressure leads to incomplete records

Consequences of skipping preparation:

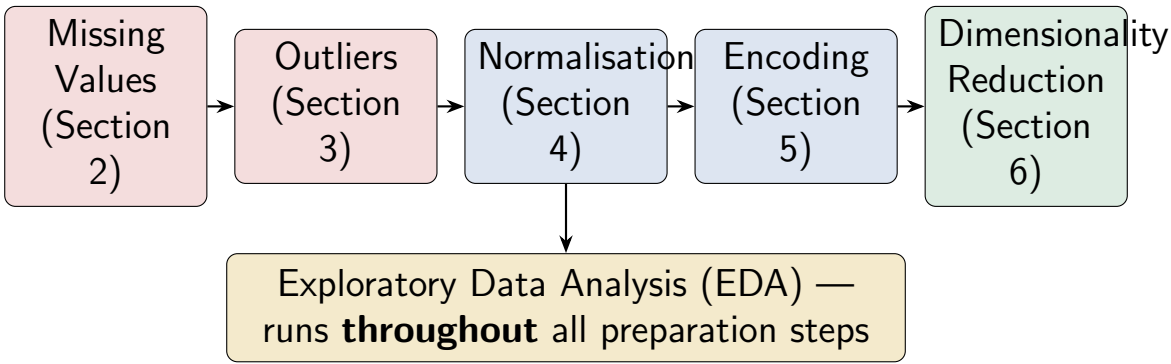
- **Decision trees** split on outliers and overfit the training data.
- **Distance measures** become meaningless when variables are on wildly different scales.

The Dirty Secret of Data Mining

Garbage In, Garbage Out

No algorithm, however sophisticated, can compensate for dirty input data.

What Preparation Involves: An Overview



What Preparation Involves: An Overview

In our health insurance dataset:

- 5% missing `bmi`; 4% missing `income`
- 10% missing `complaint_notes`
- Injected extreme BMI and income outliers

Tools we will use:

- `tidyverse` — core wrangling grammar
- `skimr` — fast summaries
- `DataExplorer` — automated EDA

What Preparation Involves: An Overview

In our health insurance dataset:

- Variables on vastly different scales (`income` vs. `age`)
- Nominal variables (`plan_tier`, `region`) need encoding

Tools we will use:

- `sparklyr` — scaling to 500,000 rows
- `recipes` — reproducible preprocessing pipelines

Starting Point: Diagnosing the Dataset

```r
library(dplyr)
library(skimr)

# Load the full dataset
source("simulate_bdat602.R")
health_data <- simulate_bdat602(n = 500000, seed = 602)

# Step 1: Comprehensive summary of all 38 variables
skim(health_data)
# Reports: n_missing, complete_rate, mean, sd, min, max,
#          histogram (numeric) and n_unique (character)
```

Starting Point: Diagnosing the Dataset

```
# Step 2: Missing value counts per variable
health_data |>
summarise(across(everything(), ~ sum(is.na(.)))) |>
tidyr::pivot_longer(everything(),
names_to  = "variable",
values_to = "n_missing") |>
filter(n_missing > 0) |>
mutate(pct_missing = round(100 * n_missing / nrow(health_
   data), 2)) |>
arrange(desc(pct_missing))
```

What to Look For

Variables with $< 50\%$ missing values require a deliberate imputation strategy. Variables with $> 50\%$ missing are candidates for removal unless they carry critical information.

Types of Missingness: Why It Matters

Not all missing values are equal. The mechanism of missingness determines what you can and cannot do about it.

1. **MCAR** — Missing Completely At Random
   The probability of being missing is unrelated to any variable (observed or unobserved).

   Safe to drop rows or impute with mean/median.

Types of Missingness: Why It Matters

1. **MAR** — Missing At Random
   The probability of being missing depends on *observed* variables, not on the missing value itself.

   Imputation using other variables is appropriate.

2. **MNAR** — Missing Not At Random
   The probability of being missing depends on the *missing value itself*.

   Imputation is biased; consider adding a missingness indicator.

Types of Missingness: Why It Matters

Our dataset examples:

Variable	Mechanism	Reason
`bmi`	MAR	Not collected for digital sign-ups
`income`	MNAR	High earners more likely to refuse disclosure
`complaint_notes`	MCAR	No complaint lodged

Test for MCAR

Use Little's MCAR test or simply check whether missingness in one variable is correlated with values in other variables.

Strategies for Handling Missing Values

Strategy	When to Use	Advantage	Risk
Complete-case (listwise deletion)	MCAR only; $<$ 5% missing	Simple; unbiased if MCAR	Loses data; biased if MAR/MNAR
Mean / Median imputation	MCAR or MAR; numeric variables	Fast; easy to implement	Reduces variance; ignores correlation
Mode imputation	MCAR or MAR; categorical variables	Fast	Overrepresents the mode

Strategies for Handling Missing Values

Strategy	**When to Use**	**Advantage**	**Risk**
Model-based (kNN, regression)	MAR; important variable	Uses all available information	Computationally expensive
Missingness indicator flag	MNAR; when absence = information	Retains signal about absence	Adds columns; increases model complexity
Drop variable	More than 50% missing; not informative	Reduces noise	Loses information if MNAR

R Application: Imputing Missing Values

```r
library(dplyr)
library(tidyr)

# --- Strategy 1: Median imputation for bmi (MAR) ---
bmi_median <- median(health_data$bmi, na.rm = TRUE)

health_clean <- health_data |>
mutate(
bmi_missing = as.integer(is.na(bmi)),   # missingness
   indicator
bmi         = if_else(is.na(bmi), bmi_median, bmi)
)
```

R Application: Imputing Missing Values

```r
# --- Strategy 2: Median imputation for income (MNAR) ---
# Add a flag first because missingness carries
   information
income_median <- median(health_data$income, na.rm = TRUE)

health_clean <- health_clean |>
mutate(
income_missing = as.integer(is.na(income)),
income         = if_else(is.na(income), income_median,
   income)
)
```

R Application: Imputing Missing Values

```
# --- Strategy 3: Complaint notes --- treat NA as "No
   complaint" ---
health_clean <- health_clean |>
mutate(
has_complaint  = as.integer(!is.na(complaint_notes)),
complaint_notes = if_else(is.na(complaint_notes),
"no complaint", complaint_notes)
)

# Verify: no more missing values in these variables
colSums(is.na(health_clean[, c("bmi", "income", "
   complaint_notes")]))
```

R Application: kNN Imputation for Better Accuracy

```
library(recipes)

# kNN imputation uses the k nearest neighbours of each
# missing record (based on observed variables) to
   estimate
# the missing value --- more accurate than median for MAR
    data
```

R Application: kNN Imputation for Better Accuracy

```
library(recipes)
impute_recipe <- recipe(claim_amount ~ ., data = health_
  data) |>
step_impute_knn(bmi,
neighbors = 5,
impute_with = imp_vars(age, sex, region,
employment_type,
plan_tier)) |>
step_impute_median(income)   # median for MNAR (faster)
# Prep on training data only (to avoid leakage)
impute_prep <- prep(impute_recipe, training = health_data
  )
```

R Application: kNN Imputation for Better Accuracy

```
# Apply to the full dataset
health_imputed <- bake(impute_prep, new_data = health_
   data)
# Check
summary(health_imputed$bmi)
```

Why recipes?

Using a `recipe` ensures that imputation parameters (e.g. the median, the k neighbours) are learned from **training data only** and applied consistently to new data. This is critical for preventing data leakage.

Handling Missing Values in Spark at Scale

```
library(sparklyr)
library(dplyr)
sc <- spark_connect(master = "local[*]")
health_tbl <- copy_to(sc, health_data,
name = "health_insurance", overwrite = TRUE)
# --- Count missing values per column in Spark ---
health_tbl |>
summarise(
bmi_missing    = sum(as.integer(is.na(bmi)),    na.rm =
   TRUE),
income_missing = sum(as.integer(is.na(income)), na.rm =
   TRUE)
)
```

Handling Missing Values in Spark at Scale

```
# --- Median imputation in Spark ---
bmi_med_spark <- health_tbl |>
summarise(med = percentile_approx(bmi, 0.5, accuracy =
   1000)) |>
collect() |>
pull(med)
health_tbl_clean <- health_tbl |>
mutate(
bmi_missing = as.integer(is.na(bmi)),
bmi = if_else(is.na(bmi), bmi_med_spark, bmi)
                  )
sdf_nrow(health_tbl_clean)    # confirm no rows dropped
```

What is an Outlier?

Definition

An **outlier** is an observation that deviates markedly from the general pattern of the data. It may represent a genuine extreme value, a measurement error, a data entry mistake, or a fraudulent record.

What is an Outlier?

Three types of outliers:

1. **Global outlier (point anomaly)**: a single value that is extreme relative to the entire dataset

   BMI = 200 in our dataset (entry error)

   A $50,000 claim for a Bronze plan is unusual; for Platinum it is not

Why outliers matter in mining:

- ***k*-means**: a single extreme point can pull an entire cluster centroid
- **Linear models**: one high-leverage point can dominate the fit.

What is an Outlier?

Three types of outliers:

1. **Contextual outlier**: a value that is extreme only within a specific context
   A $50,000 claim for a Bronze plan is unusual; for Platinum it is not.

Why outliers matter in mining:

- **Distance measures**: extreme values inflate distances and distort similarity
- **Association rules**: outlier values in continuous variables create spurious bins.

What is an Outlier?

Three types of outliers:

1. **Collective outlier**: a group of observations that are anomalous together
   A cluster of weekend claims from the same agent

Why outliers matter in mining:

- **Normalisation**: a single extreme value compresses all other data into a narrow range.

Outliers $\neq$ Errors

Sometimes outliers *are* the most interesting data points. In fraud detection, the outlier **is** the signal.

Methods for Detecting Outliers

1. **Visual methods:**
   - **Box plot**: values beyond $Q_3 + 1.5 \times \text{IQR}$ or below $Q_1 - 1.5 \times \text{IQR}$ are flagged as potential outliers
   - **Histogram**: extreme bars far from the bulk of the distribution
   - **Scatter plot**: points isolated from the main cloud
2. **Model-based detection:**
   - **Isolation Forest**: isolates anomalies by randomly partitioning the feature space; anomalies require fewer splits.

Methods for Detecting Outliers

3. Statistical rules:

- **Z-score**: $|z_i| = \left|\frac{x_i - \bar{x}}{s}\right| > 3$ flags an outlier (assumes normality)
- **IQR rule**: $x_i < Q_1 - 1.5\,\text{IQR}$ or $x_i > Q_3 + 1.5\,\text{IQR}$
- **Modified Z-score**: uses median and MAD; robust to non-normality

4. Model-based detection:

- **LOF (Local Outlier Factor)**: compares the density of a point's neighbourhood to its neighbours' neighbourhoods
- **DBSCAN**: points not belonging to any dense cluster are labelled as noise (outliers)

Methods for Detecting Outliers

Treatment options after detection:

- Remove the record (if clear error)
- Cap/Winsorise to a boundary value
- Flag with an indicator variable and retain
- Transform the variable (log, square root)

R Application: Detecting Outliers in bmi and income

```
library(ggplot2)
# --- Visual inspection: box plot of BMI ---
ggplot(health_clean, aes(y = bmi)) +
geom_boxplot(fill = "steelblue", outlier.colour = "red",
outlier.alpha = 0.4) +
labs(title = "BMI Distribution with Outliers",
y = "BMI") +
theme_minimal()
# --- IQR rule: flag BMI outliers ---
q1_bmi  <- quantile(health_clean$bmi, 0.25, na.rm = TRUE)
q3_bmi  <- quantile(health_clean$bmi, 0.75, na.rm = TRUE)
iqr_bmi <- q3_bmi - q1_bmi
```

```r
health_clean <- health_clean |>
mutate(
bmi_outlier = bmi < (q1_bmi - 1.5 * iqr_bmi) |
bmi > (q3_bmi + 1.5 * iqr_bmi)
)
health_clean |> count(bmi_outlier)
# --- Z-score for income ---
health_clean <- health_clean |>
mutate(
income_z      = (income - mean(income, na.rm = TRUE)) /
sd(income, na.rm = TRUE),
income_outlier = abs(income_z) > 3
)
health_clean |> count(income_outlier)
```

R Application: Treating Outliers — Winsorisation

```
#Winsorisation (capping): replace extremes with boundary
   values
# Cap BMI to [10, 60] --- biologically plausible range
# Cap income at the 1st and 99th percentile
p01_income <- quantile(health_clean$income, 0.01, na.rm =
    TRUE)
p99_income <- quantile(health_clean$income, 0.99, na.rm =
    TRUE)
health_clean <- health_clean |>
mutate(
bmi    = pmax(10, pmin(60, bmi)),
income = pmax(p01_income, pmin(p99_income, income))
)
```

```r
# --- Verify: re-check distributions after treatment ---
health_clean |>
select(bmi, income) |>
summary()
# --- Log-transform income (right-skewed variable) ---
# Log transformation compresses the right tail and
# makes the variable more symmetric --- better for
# distance-based algorithms (clustering, kNN)
health_clean <- health_clean |>
mutate(log_income = log1p(income))   # log1p handles zero
    safely
# Compare distributions
par(mfrow = c(1, 2))
hist(health_clean$income,     main = "Income (raw)",
        col = "steelblue")
hist(health_clean$log_income, main = "Income (log1p)",
       col = "darkorange")
```

Why Scale Matters in Data Mining

The problem: variables on incompatible scales

Consider computing the distance between two policyholders using age and `income`:

$$d = \sqrt{(\Delta\,\text{age})^2 + (\Delta\,\text{income})^2}$$

- Δ age might be 5 years (small range)
- Δ income might be \$50,000 (huge range)

Unaffected algorithms (do not require scaling):

- Decision trees and random forests (split on ranks)
- Naïve Bayes (models variables independently)
- Association rule mining (uses binary/categorical data).

Why Scale Matters in Data Mining

Income completely **dominates** the distance calculation. Age becomes irrelevant.

Affected algorithms (require scaling):

- k-means and hierarchical clustering
- k-nearest neighbours (kNN)
- PCA

Unaffected algorithms (do not require scaling):

- Decision trees and random forests (split on ranks)
- Naïve Bayes (models variables independently)
- Association rule mining (uses binary/categorical data)

Why Scale Matters in Data Mining

Affected algorithms (require scaling):

- Support Vector Machines (BDAT 604)
- Ridge and Lasso regression (BDAT 604)

Rule of Thumb

If the algorithm uses **distances**, **dot products**, or **gradient descent**, you must scale your numeric features. If it uses **splits** or **conditional probabilities**, scaling is optional.

Three Scaling Methods Compared

1. Min-Max Normalisation

$$x_i' = \frac{x_i - x_{\min}}{x_{\max} - x_{\min}}$$

Maps all values to $[0, 1]$.

Use when: bounded range needed; no extreme outliers.

Problem: one outlier compresses all other values.

2. Robust Scaling

$$x_i' = \frac{x_i - \text{Median}(x)}{\text{IQR}(x)}$$

Uses median and IQR instead of mean and SD.

Use when: data has outliers; IQR is more robust than SD.

Problem: less familiar to stakeholders.

Three Scaling Methods Compared

3. Z-Score Standardisation (Standardisation)

$$x_i' = \frac{x_i - \bar{x}}{s}$$

Centres at 0 with unit standard deviation.

Use when: distribution is roughly symmetric; most common choice.

Problem: does not bound the range.

Robust Scaling

Method	Range	Robust?
Min-Max	$[0, 1]$	No
Z-score	$(-\infty, \infty)$	No
Robust	$(-\infty, \infty)$	Yes

R Application: Scaling Numeric Variables

```
library(recipes)
# Define numeric variables to scale
num_vars <- c("age", "bmi", "income", "premium", "
   deductible",
"coverage_amount", "policy_age_months",
"num_chronic_conditions", "num_visits",
"num_prescriptions", "num_hospital_admissions",
"num_claims", "avg_past_claim", "claim_amount",
"app_logins_monthly", "support_calls")
# --- Method 1: Z-score standardisation via recipes ---
scale_recipe <- recipe(~ ., data = health_clean) |>
step_normalize(all_of(num_vars))
scale_prep   <- prep(scale_recipe, training = health_
   clean)
```

R Application: Scaling Numeric Variables

```
health_scaled <- bake(scale_prep, new_data = health_clean
   )
# Check: mean should be ~0, sd should be ~1
health_scaled |>
select(age, bmi, income) |>
summarise(across(everything(),list(mean = mean, sd = sd),
                 .names = "{.col}_{.fn}"))
# --- Method 2: Min-max scaling (manual) ---
min_max_scale <- function(x) (x - min(x, na.rm=TRUE)) /
(max(x, na.rm=TRUE) - min(x, na.rm=TRUE))
health_clean <- health_clean |>
mutate(age_mm  = min_max_scale(age),
bmi_mm  = min_max_scale(bmi))
```

Why Categorical Variables Need Encoding

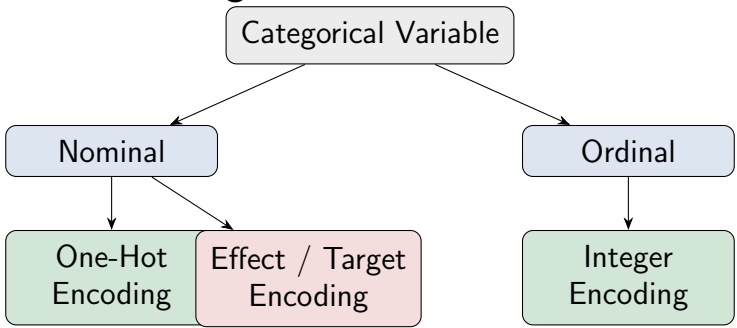
Most mining algorithms require **numeric input**. A variable like `plan_tier` with values {"Bronze", "Silver", "Gold", "Platinum"} cannot be fed directly into k-means or a distance formula.

The naive mistake: assign numbers (Bronze = 1, Silver = 2, Gold = 3, Platinum = 4).0

1. This implies Gold is *three times* Bronze — a false ordering imposed on data.
2. Algorithms using distances or arithmetic will treat this as meaningful.
3. Only valid for *ordinal* variables where the ordering is genuine.

In our dataset:

- **Nominal**: `sex`, `region`, `payment_method` — no ordering
- **Ordinal**: `education`, `plan_tier`, `customer_rating` — genuine ordering



Encoding Methods in Detail

Method	How	Best For	Advantage	Risk
One-Hot (Dummy)	k levels $\rightarrow$ $k-1$ binary columns	Nominal variables; few levels	No false ordering	Curse of dimensionality
Ordinal / Integer	Assign integers preserving order	Genuinely ordered variables	Simple; parsimonious	Magnitudes may not be equal
Target Encoding	Replace level with outcome mean	High-cardinality nominal variables	Compact; informative	Risk of data leakage
Binary Encoding	Encode as binary bit strings	High-cardinality; tree methods	Very compact	Less interpretable

Encoding Methods in Detail

Method	How	Best For	Advantage	Risk
Binary Encoding	Encode as binary bit strings	High-cardinality; tree methods	Very compact	Less interpretable
Frequency Encoding	Replace with count of level	High-cardinality; any algorithm	Simple; fast	Collisions possible

The Dummy Variable Trap

With one-hot encoding, always drop **one** category per variable (the reference level) to avoid perfect multicollinearity. `recipes` and `model.matrix()` do this automatically.

R Application: Encoding Categorical Variables

```
library(recipes)
# --- One-hot encoding for nominal variables ---
encode_recipe <- recipe(claim_amount ~ ., data = health_
   clean) |>
# Nominal: create k-1 dummy columns (drop first level)
step_dummy(sex, region, payment_method,
employment_type, one_hot = FALSE) |>
# Ordinal: integer-encode with explicit level ordering
step_integer(education,
levels = c("None","Primary","Secondary",
"Tertiary","Postgraduate")) |>
step_integer(plan_tier,
```

R Application: Encoding Categorical Variables

```
library(recipes)
levels = c("Bronze","Silver","Gold","Platinum")) |>
# Remove near-zero variance columns created by encoding
step_nzv(all_predictors())
encode_prep   <- prep(encode_recipe, training = health_
   clean)
health_encoded <- bake(encode_prep, new_data = health_
   clean)
# Inspect new column names
names(health_encoded) |> head(30)
# Verify no character columns remain
sapply(health_encoded, class) |> table()
```

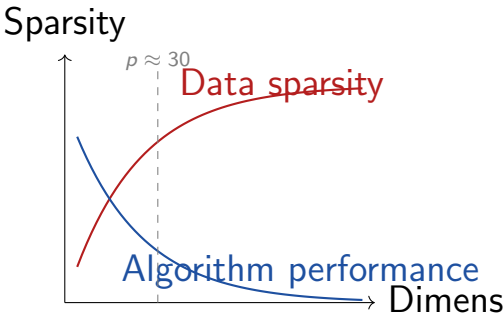
Ordinal Encoding for `plan_tier`

Treating plan tier as ordinal (Bronze $<$ Silver $<$ Gold $<$ Platinum) is reasonable here because both coverage and premium increase monotonically. Always check this assumption against domain knowledge.

The Curse of Dimensionality

The Problem

As the number of variables (dimensions) p grows, the volume of the feature space grows exponentially. Data becomes **increasingly sparse**, and distance-based algorithms break down.



The Curse of Dimensionality

Practical consequences:

- In very high dimensions, *all points are approximately equidistant* from each other — distance loses meaning
- k-means and kNN performance degrades sharply with p
- Visualisation becomes impossible beyond $p = 3$

Solution strategies:

- **Feature selection**: remove irrelevant variables

The Curse of Dimensionality

Practical consequences:

- Models overfit: many variables but few informative ones.

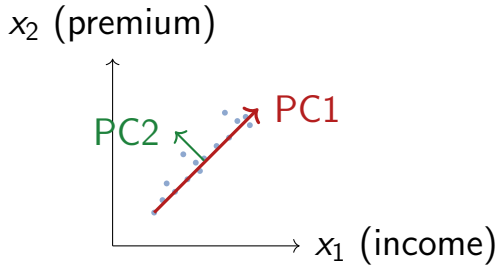
Solution strategies:

- **Feature extraction**: create a smaller set of new variables that capture most of the variance — PCA

Principal Component Analysis: The Intuition

The core idea — without algebra:

Imagine you have data scattered in 2D space. PCA finds the **directions of greatest spread** (variance) in that data:



Principal Component Analysis: The Intuition

- **PC1** is the direction along which the data varies the most
- **PC2** is the direction of the second-most variation, *perpendicular to PC1*
- Each subsequent PC captures less and less variance

Why this helps:

1. We can often keep just 2–5 PCs and retain 80–95% of the information
2. The new axes (PCs) are uncorrelated — no multicollinearity

Principal Component Analysis: The Intuition

Why this helps:

3. We can plot PC1 vs. PC2 to visualise high-dimensional data in 2D

PCA: Key Concepts Without Algebra

Principal Components (PCs):

- Each PC is a **linear combination** of the original variables
- PCs are **orthogonal** (uncorrelated) to each other
- The **loadings** tell you how much each original variable contributes to each PC.

Important limitations:

- PCA requires **numeric**, **scaled** variables — cannot be applied to raw categoricals
- PCs are **hard to interpret**: each is a mix of all original variables

PCA: Key Concepts Without Algebra

Principal Components (PCs):

- The **scores** are the new coordinates of each observation in PC space

How many PCs to keep?

- **Scree plot**: look for the "elbow" where variance explained flattens out.

Important limitations:

- PCA assumes **linear** relationships; non-linear variants exist (kernel PCA)
- Sensitive to **outliers** — robust PCA methods address this.

PCA: Key Concepts Without Algebra

Principal Components (PCs): How many PCs to keep?

- **Cumulative variance**: keep enough PCs to explain $\geq 80\%$ of total variance
- **Kaiser rule**: keep PCs with eigenvalue > 1 (each explains more than one original variable)

PCA as a Tool, Not a Goal

In BDAT 602, we use PCA primarily as a **preprocessing step** before clustering and as a **visualisation tool**. The full mathematical treatment is available in BDAT 604.

R Application: PCA on the Insurance Dataset

```
library(recipes)
library(ggplot2)
# Select numeric variables for PCA
# (already scaled and imputed)
pca_vars <- c("age", "bmi", "income", "premium", "
   deductible",
"num_chronic_conditions", "num_visits",
"num_prescriptions", "num_hospital_admissions",
"num_claims", "avg_past_claim", "claim_amount",
"app_logins_monthly", "support_calls",
"policy_age_months")
```

R Application: PCA on the Insurance Dataset

```
# --- PCA via recipes ---
pca_recipe <- recipe(~ ., data = health_clean[, pca_vars
   ]) |>
step_normalize(all_numeric_predictors()) |>
step_pca(all_numeric_predictors(), num_comp = 5)
pca_prep <- prep(pca_recipe)
pca_scores <- bake(pca_prep, new_data = NULL)
head(pca_scores)   # PC1, PC2, ..., PC5
```

R Application: PCA on the Insurance Dataset

```
# --- Variance explained ---
tidy(pca_prep, number = 2, type = "variance") |>
filter(terms == "percent variance") |>
ggplot(aes(x = component, y = value)) +
geom_col(fill = "steelblue") +
geom_line(group = 1, colour = "red") +
labs(title = "Scree Plot: Variance Explained by Each PC",
x = "Principal Component", y = "% Variance Explained") +
theme_minimal()
```

R Application: Visualising PCA Results

```
# --- PC loadings: which variables drive each component? ---
tidy(pca_prep, number = 2, type = "coef") |>
filter(component %in% c("PC1", "PC2")) |>
ggplot(aes(x = reorder(terms, value), y = value,
fill = value > 0)) +
geom_col(show.legend = FALSE) +
facet_wrap(~ component) +
coord_flip() +
scale_fill_manual(values = c("steelblue", "darkorange")) +
labs(title = "PCA Loadings: PC1 and PC2",
x = NULL, y = "Loading") +
theme_minimal()
```

R Application: Visualising PCA Results

```
# --- Scatter plot of PC1 vs PC2, coloured by plan_tier
pca_scores |> bind_cols(health_clean |> select(plan_tier)
   ) |>
ggplot(aes(x = PC1, y = PC2, colour = plan_tier)) +
geom_point(alpha = 0.15, size = 0.8) +
labs(title = "PCA Score Plot: PC1 vs PC2 by Plan Tier",
colour = "Plan Tier") +theme_minimal()
```

Interpreting the PCA Plot

If Platinum and Bronze policyholders separa te along PC1, it suggests PC1 captures a risk/premium dimension. Overlapping groups suggest the first two PCs do not cleanly differentiate plan tiers — more components may be needed.

EDA: What We Are Looking For

EDA is not just plotting.
It is a systematic investigation of:

1. **Distribution shapes**: symmetric, skewed, bimodal, heavy-tailed
2. **Central tendency and spread**: mean, median, IQR, standard deviation.

EDA questions for our dataset:

- Is `claim_amount` symmetric or right-skewed?
- Does `bmi` differ meaningfully by `region`?

EDA: What We Are Looking For

EDA is not just plotting.
It is a systematic investigation of:

1. **Missing values**: which variables, how many, what pattern
2. **Outliers**: how extreme, how many, plausible or error?

EDA questions for our dataset:

- Are smokers more concentrated in certain `plan_tier` categories?
- Is `fraud_flag` correlated with `weekend_claim`?

EDA: What We Are Looking For

EDA is not just plotting.
It is a systematic investigation of:

1. **Relationships between variables**: correlations, conditional distributions
2. **Class balance**: for classification targets — how many churned? How

EDA questions for our dataset:

- Do high-support-call customers have lower `customer_rating`?
- Is `churned` balanced (50/50) or imbalanced?

EDA: What We Are Looking For

EDA is not just plotting.
It is a systematic investigation of:

1. **Temporal patterns**: does claim frequency vary over policy age?

EDA questions for our dataset:

EDA Informs Every Downstream Step

The decisions made during EDA — what to impute, what to transform, what to encode — directly determine the quality of mining results.

R Application: Univariate EDA

```
library(ggplot2)
library(patchwork)
# --- Histogram: claim amount (continuous, right-skewed)
    ---
p1 <- health_clean |>
filter(claim_amount > 0) |>
ggplot(aes(x = claim_amount)) +
geom_histogram(bins = 60, fill = "steelblue", colour = "
    white") +
scale_x_log10(labels = scales::comma) +
labs(title = "Claim Amount (log scale)", x = "USD", y = "
    Count") +
theme_minimal()
```

R Application: Univariate EDA

```
# --- Bar chart: plan tier frequencies ---
p2 <- health_clean |>count(plan_tier) |>
ggplot(aes(x = reorder(plan_tier, -n), y = n, fill = plan
   _tier)) +
geom_col(show.legend = FALSE) +
geom_text(aes(label = scales::comma(n)), vjust = -0.3,
size = 3) + labs(title = "Policies by Plan Tier",
x = "Plan Tier", y = "Count") + theme_minimal()
# --- Combine with patchwork ---
p1 + p2
```

R Application: Bivariate EDA

```
# --- Box plot: claim amount by plan tier ---
health_clean |>
filter(claim_amount > 0) |>
ggplot(aes(x = plan_tier, y = claim_amount, fill = plan_
   tier)) +
geom_boxplot(outlier.alpha = 0.15) +
scale_y_log10(labels = scales::comma) +
labs(title = "Claim Amount by Plan Tier",
x = NULL, y = "Claim Amount (log scale)") +
theme_minimal() + theme(legend.position = "none")
```

R Application: Bivariate EDA

```
# --- Correlation heatmap (numeric variables) ---
library(DataExplorer)
health_clean |>
select(age, bmi, income, premium, deductible,
num_chronic_conditions, num_visits, num_claims,
claim_amount, support_calls, customer_rating) |>
plot_correlation(type = "continuous",
title = "Correlation Matrix: Numeric Variables")
# --- Fraud rate by employment type ---
health_clean |>group_by(employment_type) |>
summarise(fraud_rate = mean(fraud_flag) * 100,
n = n()) |> arrange(desc(fraud_rate))
```

R Application: EDA at Scale with sparklyr

```r
library(sparklyr)
library(dplyr)

# --- Summary statistics in Spark (stays distributed) ---
health_tbl |>summarise(
mean_age        = mean(age,          na.rm = TRUE),
mean_bmi        = mean(bmi,          na.rm = TRUE),
mean_claim      = mean(claim_amount, na.rm = TRUE),
median_claim    = percentile_approx(claim_amount, 0.5),
sd_claim        = sd(claim_amount,   na.rm = TRUE),
fraud_rate_pct  = mean(fraud_flag,   na.rm = TRUE) * 100,
churn_rate_pct  = mean(churned,      na.rm = TRUE) * 100
) |>collect()
```

R Application: EDA at Scale with sparklyr

```
# --- Group-wise summaries in Spark ---
health_tbl |>group_by(plan_tier) |> summarise(n= n(),
avg_premium   = mean(premium,      na.rm = TRUE),
avg_claim     = mean(claim_amount, na.rm = TRUE),
pct_fraud     = mean(fraud_flag,   na.rm = TRUE) * 100,
pct_churned   = mean(churned,      na.rm = TRUE) * 100
) |>arrange(desc(avg_claim)) |>collect()
```

Key Insight

With 500,000 rows, Spark computes these grouped summaries in seconds. The same operation in base R with a `for` loop would take minutes. Always push heavy aggregations to Spark and `collect()` only the summary.

R Application: Automated EDA Report

```r
library(DataExplorer)
# --- Full automated EDA report in one line ---
create_report(health_clean,
output_file  = "BDAT602_Lecture2_EDA.html",
output_dir   = ".",
report_title = "Health Insurance Data: EDA Report"
)
# --- Or run individual diagnostics interactively ---
# 1. Missing value profile
plot_missing(health_clean)
# 2. All continuous variable distributions
plot_histogram(health_clean, ncol = 4)
# 3. All categorical variable frequencies
plot_bar(health_clean, ncol = 3)
```

R Application: Automated EDA Report

```
# 4. Correlation heatmap
plot_correlation(health_clean |>
select(where(is.numeric)),
title = "Correlation Matrix")
# 5. QQ plots (check normality of key variables)
plot_qq(health_clean |>
select(age, bmi, income, claim_amount))
```

Class Imbalance: A Critical EDA Finding

What is class imbalance?

When a classification target has **very unequal class frequencies**, standard algorithms are biased toward the majority class.

In our dataset:

- `fraud_flag`: $\approx 3\%$ positive
  $\rightarrow$ a model predicting "no fraud" for *everyone* achieves 97% accuracy — but is useless
- `churned`: $\approx 25\%$ positive
  $\rightarrow$ moderate imbalance; manageable with appropriate metrics
- `high_risk`: $\approx 30\%$ positive
  $\rightarrow$ mild imbalance.

Class Imbalance: A Critical EDA Finding

Common remedies:

1. **SMOTE** (Synthetic Minority Oversampling Technique): generate synthetic minority class records
2. **Random oversampling**: duplicate minority class records
3. **Random undersampling**: remove majority class records
4. **Class weights**: penalise misclassifying the minority class more heavily
5. **Threshold adjustment**: lower the classification threshold from 0.5

Why Pipelines Matter

The problem with ad-hoc preprocessing:

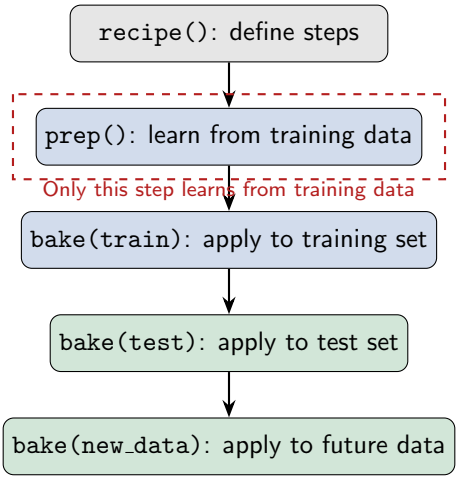
- You impute the mean of `bmi` using the *entire* dataset, including what will become the test set — this is **data leakage**
- A new record arrives for prediction: you have to manually remember every transformation applied during training
- A colleague reproduces your work: they cannot be sure the same steps were applied in the same order
- You change the training set: you have to redo all steps manually

Why Pipelines Matter

The Solution: `recipes`

A `recipe` is a **reusable, ordered sequence of preprocessing steps**. It is learned ("prepped") on training data and applied ("baked") consistently to any new data.

Why Pipelines Matter



R Application: A Complete Preprocessing Pipeline

```
library(tidymodels)
# --- Step 0: Train-test split (always first) ---
set.seed(602)
data_split  <- initial_split(health_clean, prop = 0.75,
strata = plan_tier)
train_data  <- training(data_split)
test_data   <- testing(data_split)
```

R Application: A Complete Preprocessing Pipeline

```
# --- Step 1: Define the recipe ---
prep_recipe <- recipe(claim_amount ~ ., data = train_data
  ) |>
# Imputation
step_impute_median(bmi, income) |>
step_mutate(income_missing = as.integer(is.na(income)),
bmi_missing    = as.integer(is.na(bmi))) |>
# Outlier treatment (Winsorisation)
step_mutate(bmi    = pmax(10, pmin(60, bmi)),
income = pmax(quantile(income, 0.01, na.rm = TRUE),
pmin(quantile(income, 0.99, na.rm = TRUE), income))) |>
```

R Application: A Complete Preprocessing Pipeline

```
# Log-transform skewed variables
step_log(income, claim_amount, base = 10, offset = 1) |>
# Encode categoricals
step_dummy(sex, region, payment_method, employment_type)
   |>
step_integer(education, plan_tier) |>
# Scale all numeric predictors
step_normalize(all_numeric_predictors()) |>
# Remove near-zero variance
step_nzv(all_predictors()) |>
# Remove highly correlated predictors (r > 0.95)
step_corr(all_numeric_predictors(), threshold = 0.95)
```

R Application: Prepping and Baking the Pipeline

```
# --- Step 2: Prep (learn parameters from training data only) ---
prep_fit <- prep(prep_recipe, training = train_data)
# Inspect what steps were applied and their parameters
prep_fit
# --- Step 3: Bake (apply to training and test sets) ---
train_processed <- bake(prep_fit, new_data = NULL)      # training set
test_processed  <- bake(prep_fit, new_data = test_data) # test set
```

R Application: Prepping and Baking the Pipeline

```
# --- Verify the outputs ---
dim(train_processed)  # fewer columns after nzv + corr
   removal
im(test_processed)
colSums(is.na(train_processed)) # Check: no missing
   values remain
colSums(is.na(test_processed)) # Check: all numeric (no
   character columns)
sapply(train_processed, class) |> table()
# Check: means ~0 and sds ~1 for scaled vars
train_processed |>select(age, bmi, income) |>
summarise(across(everything(),list(mean = mean,sd=sd)))
```

What We Covered Today

Conceptual foundations:

- ✓ Why data preparation consumes 80% of project time and cannot be skipped
- ✓ Three types of missingness: MCAR, MAR, MNAR — and why the type determines the remedy.

Practical (R):

- ✓ Diagnosing data with `skim()` and `DataExplorer`
- ✓ Median and kNN imputation with `recipes`
- ✓ Detecting and Winsorising outliers
- ✓ Log-transforming skewed variables.

What We Covered Today

Conceptual foundations:

- ✓ Outlier types and detection methods: visual, IQR rule, Z-score, model-based
- ✓ Normalisation: Min-Max, Z-score, Robust scaling — when each applies

Practical (R):

- ✓ Z-score standardisation and Min-Max scaling
- ✓ One-hot and ordinal encoding with `step_dummy()` and `step_integer()`
- ✓ PCA with `step_pca()` and scree plots.

What We Covered Today

Conceptual foundations:

- ✓ Categorical encoding: one-hot, ordinal, target encoding — avoiding the dummy trap
- ✓ The curse of dimensionality and PCA as a dimensionality reduction tool.

Practical (R):

- ✓ Log-transforming skewed variables
- ✓ Z-score standardisation and Min-Max scaling
- ✓ One-hot and ordinal encoding with `step_dummy()` and `step_integer()`.

What We Covered Today

Conceptual foundations:

- ✓ Class imbalance and why accuracy alone is a misleading metric.
- ✓ Why reproducible pipelines prevent data leakage.

Practical (R):

- ✓ PCA with `step_pca()` and scree plots
- ✓ Large-scale EDA with `sparklyr` group summaries
- ✓ Building a full preprocessing pipeline with `recipes`.

Looking Ahead: Lecture 3

Next Week: Finding Rules in Data — Association Mining and Market Basket Analysis

With clean, well-understood data in hand, we are ready for our first mining algorithm. Lecture 3 introduces association rule mining — discovering which variables (or products) tend to co-occur — and applies it to our insurance add-on riders.

Looking Ahead: Lecture 3

Key topics in Lecture 3:

- Support, confidence, and lift — the three rule metrics
- The Apriori algorithm step by step
- FP-Growth for large datasets (no candidate generation)

Dataset continuity:

- We use the **binary add-on rider flags**: `dental_cover`, `vision_cover`, `mental_cover`, `maternity_cover`

Looking Ahead: Lecture 3

Key topics in Lecture 3:

- Sequence patterns (ordered association rules)
- Filtering and interpreting discovered rules
- Visualising rules with `arulesViz`

Dataset continuity:

- Combined with `plan_tier`, `region`, `smoker`, and `employment_type` to form transactional records
- The cleaned data from Lecture 2 feeds directly in

R tools:

- `arules` — Apriori and FP-Growth
- `arulesViz` — rule visualisation

Key Takeaways

The Three Rules of Data Preparation

1. **Understand why values are missing before deciding how to handle them.** The mechanism (MCAR, MAR, MNAR) determines whether deletion, imputation, or flagging is appropriate.
2. **Always build a pipeline, never transform data by hand.** A `recipe` that is prepped on training data and baked on test data is the only safe way to avoid data leakage.
3. **EDA is not optional and is not a one-time step.** Return to your visualisations after each transformation to verify the change had the intended effect.

Key Takeaways

Problem	**Remedy**	**R Tool**
Missing values	Imputation (median / kNN) + flag	`step_impute_*`
Outliers	Winsorise or log-transform	`step_mutate`, `step_log`
Different scales	Standardise or normalise	`step_normalize`
Categorical variables	One-hot or ordinal encoding	`step_dummy`, `step_integer`
Too many variables	Principal Component Analysis (PCA)	`step_pca`

Self-Study and Further Reading

Core reading for Lecture 2:

- Han, Kamber & Pei (2011), *Data Mining*:
  Ch. 2: Data (types, quality, preprocessing)
  Ch. 3: Data Preprocessing (cleaning, integration, transformation)

R resources:

- Kuhn & Silge (2022), *Tidy Modeling with R*:
  Ch. 8: Feature Engineering with `recipes`
  Free at `tmwr.org`
- `recipes` documentation:
  `recipes.tidymodels.org`
- `DataExplorer` vignette:
  `cran.r-project.org/packag`

Self-Study and Further Reading

Core reading for Lecture 2:

- Tan, Steinbach & Kumar (2019):
  Ch. 2: Data (attributes, quality, preprocessing)
- Wickham & Grolemund (2017), *R for Data Science*:
  Ch. 12: Tidy Data
  Ch. 13–16: Relational data, strings, factors, dates

R resources: Practice exercises:

- Apply the full `recipes` pipeline to `health_data`; report how many columns remain after `step_nzv()` and `step_corr()`
- Compare PCA scree plots before and after Winsorising the income outliers.

Self-Study and Further Reading

Core reading for Lecture 2: Practice exercises:

- Use `sparklyr` to compute the fraud rate by `region` and `employment_type`

Questions?

Next: Lecture 3 — Finding Rules in Data:
Association Mining and Market Basket Analysis

BDAT 602 · Data Mining and Applications · University of Ghana